AI Engineer Interview Questions

LLM · RAG · LangChain · LangGraph · MCP · Prompt Engineering · System Design

Senior AI Engineer Level | 2026 Edition

---

1. LLM / GenAI Fundamentals

Q1. What is a Large Language Model (LLM)?
An LLM is a model trained on large text corpora to predict the next token and generate language-like outputs. Used for: Q&A;, summarization, extraction, reasoning-style workflows, code generation, tool-using agents.

Q2. What is a token?
A token is a chunk of text used by the model internally — a word, part of a word, punctuation, etc. Tokens matter because context window is measured in tokens, pricing is token-based, and long prompts increase latency and cost.

Q3. What is a context window?
The amount of text/tokens the model can consider in one request. Includes: system prompt, user messages, tool results, retrieved documents, and output tokens. Managing context is critical in production RAG and agent systems.

Q4. What is temperature?
Controls randomness in generation.

- Low temperature (0.0–0.3) → more deterministic — use for extraction, classification, backend tasks
- High temperature (0.7–1.0) → more diverse/creative — use for brainstorming, creative writing

Q5. What is hallucination?
When the model generates unsupported or false information confidently.

Mitigations:

- Retrieval / grounding
- Tool grounding
- Better prompting (cite your sources)
- Structured outputs
- Post-validation

Q6. What is tool calling / function calling?
Tool calling lets the model request an external function instead of answering purely from its weights. Examples: order lookup, DB query, calculator, product search, internal API call. This is the foundation of agentic AI systems.

Q7. What is structured output?
Forcing the model to return data in a predictable schema (JSON, Pydantic model, typed dict). Critical for production systems that need machine-readable outputs.

> ***Why not plain text?*** *Plain text is brittle downstream. If you need fields like intent, category, product_id, answer, confidence — structured output is much safer.*

Q8. What is an embedding?
A dense vector representation of text that captures semantic meaning. Embeddings are used for: semantic search, similarity, clustering, and retrieval in RAG systems.

Q9. Completion API vs Chat-completion API?

- Completion style — single prompt string (older pattern)

- Chat style — structured messages: system / user / assistant / tool (modern pattern)

Chat format is the standard for modern LLM apps and agent workflows.

Q10. Why not rely only on an LLM's weights for private knowledge?

- Training data has a knowledge cutoff — private/recent data won't be there
- Fine-tuning is expensive and doesn't work well for frequently changing facts
- RAG is cheaper, more traceable, updatable without retraining
- RAG can show citations / source chunks

2. RAG — Retrieval-Augmented Generation

Q11. What is RAG?

RAG = Retrieval-Augmented Generation. Flow:

- User asks a question
- Retrieve relevant documents/chunks from vector store or search engine
- Include them in the prompt context
- Model answers using those documents as grounding

Reduces hallucination and lets the model answer from private/company data.

Q12. Why use RAG instead of fine-tuning for knowledge questions?

- Easier updates — add/edit docs without retraining
- Better traceability — you can show what was retrieved
- Cheaper than retraining for frequently changing knowledge
- Can show citations / source chunks
- Fine-tuning is better for style/behavior, not fact injection

Q13. What are the main stages of a RAG pipeline?

1. Ingestion — parse and load documents
2. Chunking — split into smaller pieces
3. Embedding — convert chunks to vectors
4. Indexing — store in vector DB / search index
5. Query-time retrieval — find relevant chunks
6. Prompt assembly — inject chunks into prompt
7. LLM answer generation
8. Evaluation / monitoring

Q14. What is chunking and why does it matter?

Chunking splits documents into smaller pieces before embedding and indexing. Chunk size affects retrieval quality, context precision, token cost, and hallucination risk.

- Too large → noisy retrieval, floods the context with irrelevant text
- Too small → missing context, answers lack surrounding information

Q15. What are common chunking strategies?

- Fixed-size chunking — simple, but ignores document structure
- Recursive text splitting — respects natural boundaries
- Sentence/paragraph-aware chunking
- Heading/section-aware chunking — best for structured docs
- Semantic chunking — splits where topic changes

Section-aware chunking often outperforms naive fixed-size for real documents.

Q16. What is a vector database?

Stores embeddings and enables similarity search over them.

- ChromaDB (local/lightweight)
- Pinecone
- Weaviate

- Qdrant
- pgvector (PostgreSQL extension)
- OpenSearch with vector plugin

Q17. Keyword search vs vector search vs hybrid search?

- Keyword / BM25 — exact term matching, great for specific product IDs or names
- Vector similarity — semantic meaning matching, great for concept search
- Hybrid — combines both, usually the best in production

Hybrid search captures both exact terms and semantic similarity — use this in production.

Q18. What is reranking in RAG?

A second-stage step that reorders retrieved documents using a stronger cross-encoder model or relevance scorer.

Typical flow: *retrieve top 20 cheaply → rerank → send top 5 to LLM. Improves answer quality significantly.*

Q19. What are common failure modes in RAG?

- Poor chunking — wrong size or no structure awareness
- Bad metadata filtering — wrong tenant, wrong document type
- Wrong retrieval query — user phrasing doesn't match document phrasing
- Top-k too small (misses) or too large (floods context)
- Stale documents — index not updated after content changes
- No citation discipline — model ignores retrieved context
- Prompt injection from retrieved documents

Q20. How do you evaluate a RAG system?

Retrieval metrics:

- recall@k — was the gold chunk retrieved?
- precision@k — were retrieved chunks relevant?

Answer metrics (RAGAS framework):

- Faithfulness — is the answer grounded in retrieved context?
- Answer relevancy — does the answer address the question?
- Context recall — did retrieval find the right chunks?
- Context precision — were retrieved chunks used correctly?
- Hallucination rate — how often does the model go off-context?

Build a labeled evaluation set and run it regularly — not just once.

3. LangChain

Q21. What is LangChain?

A framework for building LLM-powered applications with components for: prompts, models, tools, output parsing, retrievers, agents, and orchestration. In 2026, LangChain is the **high-level developer framework** while LangGraph is the **stateful orchestration runtime** for complex agent workflows.

Q22. What are the core building blocks in LangChain?

- Models — LLM / chat model wrappers
- Prompts — templates and system prompts
- Tools — callable functions exposed to the model
- Output parsers — convert model text to structured output
- Retrievers — abstraction over vector stores
- Runnables / LCEL pipelines — modern composition layer
- Agents — model-driven dynamic tool selection

Q23. What is LCEL?

LCEL = LangChain Expression Language. The modern LangChain way to compose pipelines using the Runnable interface.

```
# Compose a chain: prompt → model → output parser
chain = prompt | model | parser

# Invoke it
result = chain.invoke({'question': 'What is RAG?'})
```

Each step is a Runnable — composable, streamable, batchable, testable.

Q24. What is a Runnable in LangChain?

A composable unit that can be: invoked, streamed, batched, and chained with other runnables. Makes pipelines modular and testable. Key types: RunnablePassthrough, RunnableParallel, RunnableLambda.

Q25. Chain vs Agent in LangChain?

Chain — Fixed sequence — deterministic, you define the flow. Use when the flow is known.

Agent — Model decides which tool/action to take next — dynamic, flexible but less deterministic. Use when the model must choose among tools.

Q26. What is output parsing in LangChain?

Converts raw model text into structured forms (JSON, Pydantic models, typed objects). Critical for production because free text is hard to trust programmatically.

```
from langchain_core.output_parsers import PydanticOutputParser
from pydantic import BaseModel

class Answer(BaseModel):
    answer: str
    confidence: float

parser = PydanticOutputParser(pydantic_object=Answer)
chain = prompt | model | parser
```

Q27. What is LangSmith?

LangChain's observability and evaluation platform. Use for: traces, prompt inspection, tool call inspection, debugging agent runs, dataset evaluation, regression testing.

- Trace every LLM call, tool call, retrieval step
- Inspect token usage and latency per step
- Build labeled datasets for offline evaluation
- Run regression tests before deploying prompt changes

Q28. How do you implement a custom retriever in LangChain?

```
from langchain_core.retrievers import BaseRetriever
from langchain_core.documents import Document

class MyRetriever(BaseRetriever):
    def _get_relevant_documents(self, query: str) -> list[Document]:
        # custom retrieval logic
        results = my_vector_store.search(query, k=5)
        return [Document(page_content=r.text, metadata=r.meta) for r in results]
```

Q29. What is hybrid search in LangChain?

Combining dense (vector) and sparse (BM25/keyword) retrieval. Use EnsembleRetriever to combine retrievers with weights.

```
from langchain.retrievers import EnsembleRetriever

ensemble = EnsembleRetriever(
    retrievers=[bm25_retriever, vector_retriever],
    weights=[0.4, 0.6]
)
```

Q30. When would you NOT use LangChain?

- The app is just one or two simple LLM calls — raw SDK is cleaner
- You want minimal abstraction and complete direct SDK control
- Avoiding framework churn in a very small short-lived project

4. LangGraph

Q31. What is LangGraph?

A framework/runtime for building **stateful, graph-based agent workflows** with: explicit state, nodes, edges, loops, retries, human-in-the-loop, and checkpointing/resume.

Q32. LangChain vs LangGraph — what's the difference?

LangChain — High-level framework — prompts, tools, retrieval, integrations, agent APIs

LangGraph — Orchestration runtime — stateful execution, branching, loops, checkpointing, long-running workflows

Mental model: LangChain gives the components; LangGraph gives the workflow engine.

Q33. What are nodes, edges, and state in LangGraph?

Node — A step in the graph — reads state, performs work (LLM call, tool call, routing), returns state updates

Edge — Defines how execution moves between nodes — can be fixed or conditional

State — Shared data object flowing through the graph — messages, retrieved docs, tool outputs, retry count, approval flags

Q34. What is a conditional edge?

Routes workflow depending on current state:

```
def route(state):
    if state['intent'] == 'billing':
        return 'billing_node'
    elif state['tool_failed']:
        return 'retry_node'
    elif state['needs_approval']:
        return 'human_approval_node'
    return 'answer_node'

graph.add_conditional_edges('classify_node', route)
```

Q35. What is checkpointing in LangGraph?

Persisting workflow state so execution can: resume after failure, pause for human approval, survive process restarts, and support long-running workflows. One of the biggest production advantages of LangGraph.

```
from langgraph.checkpoint.redis import RedisSaver

checkpointer = RedisSaver.from_conn_string('redis://localhost:6379')
graph = graph.compile(checkpointer=checkpointer)
```

Q36. What is human-in-the-loop in LangGraph?

The graph can pause and wait for human approval or input.

- Refund above a threshold
- Approve outbound email before sending
- Approve database change
- Verify low-confidence classification

Implemented using interrupt_before / interrupt_after on nodes.

Q37. Why is LangGraph useful for agent loops?

Many real agents are iterative. LangGraph makes the loop explicit and controllable:

- think / classify
- choose tool
- call tool
- inspect result
- call another tool if needed
- produce answer
- escalate or retry

Q38. What is the difference between supervisor and swarm patterns?

Supervisor — One orchestrator agent routes tasks to specialized sub-agents. Central control. Good for clear separation of domains.

Swarm — Agents communicate peer-to-peer, passing context between each other. More distributed. Good for emergent collaboration.

Q39. How do you prevent infinite loops in LangGraph?

- Add retry_count to state and check max retries in conditional edge
- Use recursion_limit on graph compilation
- Design conditional edges to always have an END path
- Add timeout or step count guards

Q40. When would you choose LangGraph over a plain agent loop?

- Stateful execution across multiple turns
- Retries and branching on tool failure
- Long-running workflows (minutes/hours)
- Checkpointing and resume
- Human approval gates
- Testable, explicit orchestration you can reason about

5. MCP — Model Context Protocol

Q41. What is MCP?

MCP = Model Context Protocol. An open protocol for connecting AI clients/agents to external tools, resources, and prompts in a standardized way. By 2026 it's widely adopted as the common tool transport layer across frameworks and clients.

Q42. What problem does MCP solve?

Without MCP, every tool integration is custom:

- Custom schemas per tool
- Custom auth handling
- Custom discovery
- Custom client logic

MCP standardizes how AI systems discover and use external capabilities.

Q43. What is an MCP server?

Exposes capabilities like: tools (actions), resources/documents, prompt templates/context.

- Filesystem server
- GitHub server
- DB query server
- Jira / CRM server
- Internal knowledge server

Q44. MCP vs function calling — what's the difference?

Function calling — Local tool invocation — tool defined in app runtime, model requests it in-process

MCP — Standard protocol for tool/resource integration — reusable external providers, works across frameworks/clients

Think of function calling as local invocation; MCP as a standardized integration layer.

Q45. What can MCP expose beyond tools?

- Tools — actions / functions the model can invoke
- Resources — files, docs, context the model can read
- Prompts — reusable prompt / context templates

Q46. Why is MCP useful in enterprise AI systems?

Enterprises need access to many external systems:

- GitHub, Confluence, Jira
- CRM systems
- Internal databases
- Support systems
- Internal documentation

MCP standardizes those integrations so the AI app doesn't hardcode each connector differently.

Q47. When would you choose MCP over custom tool integration?

- You have many external systems to integrate
- You want reusable standardized connectors
- Multiple clients/agents should share the same tool ecosystem

- You want tool discovery separated from app logic

For a tiny app with two simple tools, custom tools may be simpler.

Q48. What are the security considerations with MCP?

- Auth still needs careful design per server/tool
- Tool permissions — not every agent should call every tool
- Input validation on MCP server side
- Audit logging for all tool calls
- Rate limiting and confirmation steps for risky actions

Q49. How would MCP fit into an AI agent architecture?

```
# Example stack:
# FastAPI app
#   ■■■ LangGraph agent runtime
#         ■■■ MCP client layer
#               ■■■ GitHub MCP server
#               ■■■ Docs MCP server
#               ■■■ DB MCP server
#               ■■■ Jira MCP server

# Agent uses MCP-provided tools/resources
# rather than app-specific custom connectors
```

Q50. LangChain vs LangGraph vs MCP — when do you use each?

LangChain — High-level building blocks — prompts, tools, retrievers, structured outputs, agent APIs

LangGraph — Stateful orchestration — loops, branching, checkpointing, human approval, durable execution

MCP — Standardized reusable access to external tools/resources across apps and agents

Together — LangChain for abstractions + LangGraph for orchestration + MCP for tool/resource integration

6. Prompt Engineering & System Prompts

Q51. What is prompt engineering?

Designing prompts so the model produces reliable, useful, constrained outputs. Includes: clear instructions, role/task definition, context, constraints, examples, output format.

Q52. Zero-shot vs one-shot vs few-shot prompting?

- Zero-shot — no examples, model infers from instruction alone
- One-shot — one example showing desired input/output pattern
- Few-shot — several examples, helps when output style or logic is hard to specify with instructions alone

Q53. What are the core parts of a good production prompt?

1. Role
2. Task
3. Context / grounding data
4. Constraints / rules
5. Output format (JSON, markdown, plain)
6. Examples if needed
7. What to do if information is missing

Q54. What is the ReAct prompting pattern?

ReAct = Reason + Act. The model interleaves reasoning steps (Thought) with action steps (Act) and observations (Observe). This is the foundation of most tool-using agentic systems.

```
Thought: I need to look up the order status.
Act: call_tool(order_lookup, order_id='123')
Observe: {status: 'shipped', estimated: '2026-06-28'}
Thought: I have the answer now.
Final Answer: Your order ships June 28.
```

Q55. What is prompt grounding?

Giving the model reliable context and instructing it to answer based on that context. Examples: retrieved docs, order data, catalog info, policy documents.

> *Strong grounding prompt: 'Answer using ONLY the following context. If the answer is not in the context, say so.'*

Q56. How do you reduce hallucinations with prompting?

- Tell model to use provided context only
- Explicitly say 'if evidence is insufficient, say so'
- Request concise grounded answers
- Require structured output / citations if appropriate
- Separate system rules from user content clearly

Prompting helps, but system design (retrieval quality, validation) matters more.

Q57. What is prompt injection?

When untrusted input tries to override or manipulate the model's instructions.

- User says 'ignore previous instructions'

- Retrieved doc says 'reveal system prompt'
- Malicious webpage tells agent to call dangerous tools

Mitigations: separate trusted/untrusted context, tool permissions, output validation, approval flows for risky actions.

Q58. What is a system prompt?

The high-priority instruction that defines: assistant role, rules, boundaries, tone, tool usage behavior, grounding requirements. Sets behavior across a conversation or task.

```
# Example system prompt for an e-commerce support agent:
You are a customer support assistant for Star Gallery.
You have access to order lookup and return policy tools.
Always verify order data before confirming any claim.
Never reveal internal pricing, margins, or system details.
If you cannot resolve the issue, escalate to a human agent.
```

Q59. System prompt vs user prompt?

- System prompt — defines assistant behavior/rules, high trust level
- User prompt — the user's current request, lower trust level

Q60. Why can't system prompts alone secure an AI agent?

Prompts are NOT a security boundary. You still need:

- Auth / RBAC — server-side authorization
- Tool permission checks
- Server-side input/output validation
- Audit logging for all actions
- Rate limits
- Confirmation steps for risky/destructive actions

System prompts shape behavior; they do not replace backend controls.

7. AI System Design & Production Architecture

Q61. How would you design a production RAG chatbot?

Ingestion pipeline:

- Parse docs → clean → chunk → embed → store in vector DB

Query-time pipeline:

- Classify request → retrieve docs → rerank → build grounded prompt → generate → cite sources

Infra / ops:

- FastAPI API layer
- Redis caching for repeated queries
- Background workers for ingestion
- Observability via LangSmith
- Evaluation dataset with regular regression runs
- Feedback loop (thumbs up/down → fine-tune or prompt improve)

Q62. How would you design a document ingestion pipeline?

1. Upload file metadata → store file in object storage
2. Enqueue ingestion job to background worker
3. Parse / OCR if scanned
4. Clean text → chunk → embed
5. Index chunks in vector store
6. Store status + metadata in DB
7. Allow reprocessing if chunking strategy changes

States: uploaded → queued → processing → indexed → failed

Q63. How would you reduce latency in an AI application?

- Retrieve fewer but better chunks (better chunking + reranking)
- Cache common queries / results in Redis
- Use smaller model for classification/routing
- Parallelize independent calls (retrieval + user context fetch)
- Stream responses — don't wait for full completion
- Avoid unnecessary agent loops with better routing
- Use structured prompts with less boilerplate

Q64. How would you reduce cost in an LLM system?

- Trim conversation history — only keep recent N turns
- Use cheaper model for simple tasks (routing, classification)
- Cache repeated results with Redis TTL
- Reduce retrieved context size with better reranking
- Summarize old chat history instead of passing all messages
- Routing: small model first, big model only when needed
- Batch embeddings where possible

Q65. How do you handle conversation memory in production?

Separate memory into layers:

Short-term — Recent messages + active task state — keep in Redis or in-context

Long-term — Durable user preferences, past tasks, facts with provenance — only if truly needed

External knowledge — RAG documents / indexed corpora — retrieved on demand

Don't treat memory as one giant text blob — it bloats context and reduces quality.

Q66. How do you evaluate an AI system beyond 'it feels good'?

Offline evaluation:

- Benchmark dataset with expected answers
- Retrieval correctness (recall@k)
- Formatting compliance
- Hallucination checks

Online evaluation:

- Latency p50/p95
- Cost per query
- Failure rate
- User feedback (thumbs up/down)
- Task completion rate
- Escalation rate

Q67. What are the main failure modes of AI agents in production?

- Hallucinated tool assumptions — model calls tool with wrong arguments
- Poor tool selection — model picks irrelevant tool
- Looping / retry storms — agent stuck in a cycle
- Stale or wrong retrieved context
- Prompt injection from retrieved documents or user input
- Lack of observability — can't debug what happened
- State corruption in long-running workflows
- Unclear boundary between model logic and backend logic

Q68. How would you design a safe AI agent for actions like refunds or DB changes?

1. System prompt with explicit tool usage rules
2. Server-side authorization (auth / RBAC)
3. Tool allowlist — agent can only call permitted tools
4. Argument validation before execution
5. Confirmation step for destructive/high-value actions
6. Human approval for high-risk actions (LangGraph interrupt)
7. Audit logs and full traces for every action
8. Idempotency / rollback strategy where possible

Q69. How do you handle prompt injection from retrieved documents?

- Clearly separate retrieved context from system instructions in the prompt
- Treat retrieved content as user-trust level, not system-trust level
- Use structured prompt templates that isolate context sections
- Validate tool arguments before execution regardless of model output
- Add output validation layer for critical actions

Q70. How would you choose between raw OpenAI SDK vs LangChain vs LangGraph?

```
# Raw SDK
# Use when: 1-2 LLM calls, minimal abstraction needed, full control

# LangChain
# Use when: structured pipelines, retrieval, tool calling,
#           output parsing, multiple integrations

# LangGraph
# Use when: stateful workflows, loops, retries,
#           checkpointing, human approval, long-running agents
```

Q71. What is multi-agent orchestration and when would you use it?

Splitting tasks across multiple specialized agents vs one generalist agent.

- Use when tasks are clearly separable by domain (billing, shipping, returns)
- Use when a single agent context window becomes too large
- Use when you want fault isolation between agent types
- Use supervisor pattern for central routing or swarm for peer-to-peer

Q72. How do you ensure observability in a production agentic workflow?

- Use LangSmith for tracing — captures every node, tool call, LLM call
- Assign correlation IDs to each conversation/run
- Log state snapshots at each node transition
- Track token usage and latency per step
- Alert on retry storms, high error rates, high token spend

Q73. How would you build a shopping/support AI assistant?

Capabilities: product search, product detail, order status, return policy, recommendations

Architecture:

- FastAPI endpoint layer
- LangGraph agent or router layer
- RAG for policy/docs (ChromaDB or OpenSearch)
- Tools for order/product DB lookups
- Redis for session state
- LangSmith tracing + feedback collection

Q74. How do you test an agentic system end-to-end?

- Unit test each node in isolation with mock state
- Integration test with mock LLM and mock tools
- End-to-end test with real LLM against a labeled eval dataset
- Use LangSmith datasets for regression testing
- Test human-in-the-loop interrupt/resume paths
- Test failure paths: tool errors, retry exhaustion, escalation

Q75. How do you handle large documents that exceed the LLM context window?

- Chunk and retrieve only relevant sections (RAG)
- Hierarchical summarization — summarize sections, then summarize summaries
- Map-reduce pattern — process chunks independently, combine results
- Query-focused extraction — extract only relevant paragraphs before sending

- Use models with larger context windows for specific document-heavy tasks

8. Top 25 Priority Questions

If short on time, master these first:

LLM / RAG

- ■ What is hallucination and how do you mitigate it?
- ■ What is an embedding?
- ■ Walk me through a full RAG pipeline
- ■ What is chunking and what strategy do you use?
- ■ What is reranking?
- ■ How do you evaluate a RAG system? (RAGAS metrics)

LangChain / LangGraph

- ■ LangChain vs LangGraph — what's the difference?
- ■ What is LCEL / Runnable?
- ■ Chain vs Agent — when do you use each?
- ■ What is LangSmith used for?
- ■ What is state in LangGraph?
- ■ What is checkpointing?
- ■ What is human-in-the-loop?

MCP

- ■ What is MCP and what problem does it solve?
- ■ MCP vs function calling — key difference

Prompt Engineering / System Prompts

- ■ What is prompt grounding?
- ■ What is prompt injection and how do you defend against it?
- ■ System prompt vs user prompt
- ■ Why system prompts alone don't secure an agent

System Design

- ■ Design a production RAG chatbot
- ■ Design a document ingestion pipeline
- ■ How do you reduce LLM latency?
- ■ How do you reduce LLM cost?
- ■ How do you safely design an agent for destructive actions?

9. Interview Prep Roadmap

Phase 1 — Backend Foundation

- Python: async/await, GIL, decorators, generators
- FastAPI: dependency injection, streaming, project structure
- PostgreSQL / Redis / async patterns

Phase 2 — LLM Fundamentals

- Tokens, context windows, temperature
- Tool calling and structured outputs
- Embeddings and semantic search
- Hallucination and mitigation strategies

Phase 3 — RAG

- Full ingestion → retrieval → generation pipeline
- Chunking strategies
- Vector search vs hybrid search
- Reranking
- RAGAS evaluation framework

Phase 4 — Agent Stack

- LangChain: LCEL, tools, output parsers, retrievers
- LangGraph: state, nodes, edges, checkpointing, HITL
- LangSmith: tracing and evaluation
- MCP: servers, clients, tool vs resource

Phase 5 — Production Concerns

- Tracing and observability
- Cost optimization patterns
- Safety and prompt injection defense
- Caching and latency reduction
- Memory architecture (short / long-term / retrieval)
- Human approval flows for risky actions

> *For your profile (8+ years, production RAG, LangGraph, agentic systems), interviewers will ask you to walk through the architecture of your actual projects. Be ready to explain design decisions: why LangGraph over a plain loop, how you scoped tool use, how you measured your improvements (40% ticket reduction, 71%→89% accuracy), and what you'd do differently.*